

Aplicações para Internet

Aula 7 – Layout Moderno e Responsividade

Stack: HTML5 / CSS3

Carga Horária: 2 horas

Prof. Jeofton Costa

Conteúdo desta Aula

1. Flexbox — Distribuição e Alinhamento Unidimensional
2. CSS Grid — Layout Bidimensional e Template Areas
3. Media Queries — Breakpoints e Estratégia Mobile-First
4. Unidades de Medida CSS — Absolutas e Relativas
5. Prática — Tornar o Layout do Projeto Responsivo

ETAPA TEÓRICA — 60 minutos

1. Nivelamento — Conectando com a Aula Anterior

Nas aulas anteriores você aprendeu a estruturar documentos HTML semânticos e a estilizá-los com CSS fundamental: seletores, box model, especificidade e cascata. Agora chegamos ao coração do layout moderno: como distribuir elementos no espaço da tela de forma eficiente, flexível e responsiva.

A web de hoje é acessada por dispositivos com larguras que variam de 320 px (smartphones antigos) a mais de 2 560 px (monitores ultrawide). Construir interfaces que funcionem bem em toda essa gama é um requisito profissional inegociável — e é exatamente o que Flexbox, CSS Grid e Media Queries resolvem de maneira elegante e padronizada.

Conexão com o mercado

Segundo o relatório State of CSS 2023, Flexbox e Grid são as duas ferramentas de layout mais usadas por desenvolvedores front-end profissionais no mundo — acima de float, table-layout e qualquer framework CSS.

2. Flexbox — Layout Unidimensional

2.1 O Modelo Mental

Flexbox (Flexible Box Layout Module) opera em um único eixo por vez: horizontal (row) ou vertical (column). Ele resolve com elegância os problemas que float e inline-block nunca resolveram bem: centralização vertical, distribuição de espaço igual entre filhos e reordenação visual sem alterar o HTML.

O Flexbox possui dois protagonistas: o flex container (o elemento pai que recebe display: flex) e os flex items (os filhos diretos desse container). As propriedades se dividem entre os dois papéis — confundir onde cada propriedade vai é o erro mais comum entre iniciantes.

2.1.1 Eixos e Direção

Todo container flex possui um eixo principal (main axis) e um eixo cruzado (cross axis). A propriedade flex-direction define qual é qual:

```
/* Direção padrão: itens em linha (esquerda → direita) */
.container {
  display: flex;
  flex-direction: row;      /* padrão */
  /* outros valores: row-reverse | column | column-reverse */
}
```

2.1.2 Distribuição no Eixo Principal — *justify-content*

```
.container {
  display: flex;
  justify-content: flex-start; /* padrão – itens agrupados no início */
  /* justify-content: flex-end;    – agrupados no final */
  /* justify-content: center;      – centralizados */
  /* justify-content: space-between; – espaço igual ENTRE os itens */
  /* justify-content: space-around; – espaço igual AO REDOR dos itens */
  /* justify-content: space-evenly; – espaço exatamente igual em todos */
}
```

2.1.3 Alinhamento no Eixo Cruzado — *align-items*

```
.container {
  display: flex;
  align-items: stretch; /* padrão – itens ocupam toda altura do container */
  /* align-items: flex-start; – alinhados ao topo */
  /* align-items: flex-end;   – alinhados ao fundo */
  /* align-items: center;     – centralizados verticalmente */
  /* align-items: baseline;   – alinhados pela linha de base do texto */
}
```

☒ Centralização perfeita em duas linhas

O problema mais pesquisado no CSS por anos — centralizar um elemento horizontal E verticalmente — resolve-se com Flexbox:

```
.wrapper { display: flex; justify-content: center; align-items: center; }
```

Isso é tudo. Sem hacks com margin: auto ou position: absolute.

2.1.4 Flex Items — *Crescimento, Encolhimento e Tamanho Base*

As propriedades flex-grow, flex-shrink e flex-basis controlam como cada item se comporta quando há espaço sobrando ou faltando. O shorthand flex é a forma recomendada de usá-las:

```
.item {
  /* flex: <grow> <shrink> <basis> */
  flex: 1; /* equivale a flex: 1 1 0% – item cresce e encolhe igualmente */
  /* flex: 0 0 200px; /* largura fixa de 200px – não cresce nem encolhe */
  /* flex: 2; /* cresce o dobro dos itens com flex: 1 */
}
```

2.1.5 Quebra de Linha — *flex-wrap*

Por padrão, itens flex tentam caber em uma única linha (nowrap), o que pode causar overflow. Use flex-wrap: wrap para permitir que itens se movam para a próxima linha quando não couberem:

```
.container {
  display: flex;
  flex-wrap: wrap;          /* itens quebram para a próxima linha */
  gap: 16px;                /* espaçamento entre itens (row-gap + column-gap) */
}

.card {
  flex: 1 1 280px;         /* cresce, encolhe, base de 280px – responsivo natural!
  */
}
```

Exemplo Real: Barra de Navegação Responsiva

```
<nav class="navbar">
  <a class="logo">MeuSite</a>
  <ul class="nav-links">
    <li><a>Início</a></li>
    <li><a>Sobre</a></li>
    <li><a>Contato</a></li>
  </ul>
</nav>
```

```
.navbar {
  display: flex;
  justify-content: space-between; /* logo na esquerda, links na direita */
  align-items: center;           /* centralização vertical */
  padding: 0 2rem;
  height: 64px;
}

.nav-links {
  display: flex;
  gap: 1.5rem;
  list-style: none;
}
```

2.2 Propriedades do Flex Item

Propriedade	Valores Comuns	O que faz
align-self	auto flex-start center flex-end	Sobrescreve align-items para um item específico

order	inteiro (padrão: 0)	Altera a ordem visual do item sem mudar o HTML
flex-grow	número ≥ 0 (padrão: 0)	Quanto o item cresce em relação aos irmãos
flex-shrink	número ≥ 0 (padrão: 1)	Quanto o item encolhe quando falta espaço
flex-basis	auto tamanho CSS	Tamanho base antes de aplicar grow/shrink

3. CSS Grid — Layout Bidimensional

3.1 Por que Grid?

Se o Flexbox cuida de layouts unidimensionais (uma linha ou uma coluna), o CSS Grid resolve o problema bidimensional: linhas E colunas ao mesmo tempo. Pense no Grid como uma planilha invisível sobre a qual você posiciona seus elementos com precisão cirúrgica.

O Grid é ideal para a estrutura macro de uma página (header, sidebar, main, footer) enquanto o Flexbox brilha nos componentes internos (itens de um card, botões de um grupo, etc.). Profissionais usam ambos em conjunto — não é uma questão de escolher um ou outro.

3.2 Definindo uma Grade

```
.container {
  display: grid;

  /* Define 3 colunas: 2 fixas de 200px + 1 que ocupa o espaço restante */
  grid-template-columns: 200px 200px 1fr;

  /* Alternativa com repeat(): 3 colunas iguais */
  grid-template-columns: repeat(3, 1fr);

  /* Define 2 linhas explícitas */
  grid-template-rows: 80px auto;

  /* Espaçamento entre células */
  gap: 24px;          /* equivale a row-gap: 24px + column-gap: 24px */
}
```

A unidade fr (fraction)

fr é exclusiva do CSS Grid e representa uma fração do espaço disponível APÓS alocar tamanhos fixos e automáticos.

grid-template-columns: 250px 1fr 2fr

→ 250px fixo para sidebar, e o restante dividido: conteúdo central recebe 1/3 e conteúdo principal recebe 2/3.

Isso seria extremamente difícil de replicar com Flexbox ou float.

3.3 Grid Template Areas

A propriedade `grid-template-areas` permite nomear regiões da grade usando uma sintaxe visual que espelha o layout real. Isso torna o código CSS autoexplicativo e facilita manutenção:

```
.layout {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-rows: 64px 1fr 48px;
  grid-template-areas:
    "header header"
    "sidebar main "
    "footer footer";
  min-height: 100vh;
}

/* Posicionando os filhos nas áreas nomeadas */
.header { grid-area: header; background: #1F4E79; }
.sidebar { grid-area: sidebar; background: #F0F4F8; }
.main { grid-area: main; padding: 2rem; }
.footer { grid-area: footer; background: #E8ECF0; }
```

3.4 Posicionamento Explícito

Além das áreas nomeadas, você pode posicionar um item especificando as linhas de grade exatas que ele deve ocupar:

```
.destaque {
  /* Ocupa da coluna 1 até a coluna 3 (span de 2 colunas) */
  grid-column: 1 / 3;
  /* Equivalente mais legível: */
  grid-column: 1 / span 2;

  /* Ocupa da linha 2 até a linha 4 */
  grid-row: 2 / 4;
}
```

3.5 Grid Implícito e auto-fill / auto-fit

Para grades com quantidade de itens desconhecida (listas de cards, galerias), `auto-fill` e `auto-fit` são ferramentas poderosas que criam layouts responsivos sem uma única media query:

```
.galeria {
  display: grid;
```

```
/* Cria tantas colunas de no mínimo 280px quanto couberem na largura */
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 1.5rem;
}

/* auto-fill: mantém colunas vazias se não houver itens suficientes */
/* auto-fit: colapsa colunas vazias, itens existentes crescem para preencher */
```

Comparação: Flexbox vs CSS Grid

Use FLEXBOX quando:

- Distribuir itens em 1 eixo
- Componentes (nav, card, botões)
- Ordem e alinhamento de conteúdo
- Wrap orgânico de itens

Use CSS GRID quando:

- Layout 2D (linha + coluna)
- Estrutura macro da página
- Sobreposição de elementos
- Áreas nomeadas complexas

Na prática: Grid para a estrutura, Flexbox para os componentes.

4. Media Queries — Responsividade Estratégica

4.1 O que são Media Queries

Media Queries são condicionais CSS que aplicam blocos de estilo apenas quando certas características do dispositivo são satisfeitas — largura de viewport, orientação, resolução, tipo de dispositivo, entre outras. Elas são o mecanismo fundamental de Responsive Web Design (RWD), conceito popularizado por Ethan Marcotte em 2010.

```
/* Estrutura básica */
@media (condição) {
  /* Estilos aplicados apenas quando a condição for verdadeira */
  seletor { propriedade: valor; }
}

/* Exemplos de condições */
@media (max-width: 768px) { ... } /* largura máxima */
@media (min-width: 1024px) { ... } /* largura mínima */
@media (orientation: landscape) { ... } /* modo paisagem */
@media (prefers-color-scheme: dark) { ... } /* modo escuro */
@media (hover: hover) { ... } /* dispositivo com mouse */
```

4.2 Abordagem Mobile-First

A estratégia Mobile-First define os estilos base para telas menores e usa min-width para adicionar complexidade progressivamente. Isso resulta em CSS mais limpo, melhor

performance em mobile (menos CSS para sobrescrever) e alinhamento com a realidade do tráfego web atual — onde mais de 60% das visitas são móveis.

```
/* ✗ Desktop-First (má prática) – começa grande, desfaz para mobile */
.card { display: grid; grid-template-columns: 1fr 2fr; }
@media (max-width: 768px) {
  .card { display: block; } /* precisa sobrescrever */
}

/* ✔ Mobile-First (boa prática) – começa simples, adiciona progressivamente */
.card { display: block; }
@media (min-width: 768px) {
  .card { display: grid; grid-template-columns: 1fr 2fr; }
}
```

4.3 Breakpoints Estratégicos

Breakpoints são os pontos de largura onde o layout muda. O erro mais comum é usar breakpoints baseados em dispositivos específicos ("iPhone 14 tem 390px"). A abordagem correta é usar breakpoints baseados no conteúdo: onde o layout começa a quebrar, você adiciona um breakpoint.

Breakpoint	Valor sugerido	Dispositivos	Mudanças típicas de layout
xs (base)	< 480px	Smartphones pequenos	Coluna única, fonte reduzida, nav colapsada
sm	>= 480px	Smartphones grandes	Cards em 2 colunas, padding aumentado
md	>= 768px	Tablets, phablets	Sidebar aparece, grid de 3 colunas
lg	>= 1024px	Laptops	Layout completo, tipografia aumenta
xl	>= 1280px	Desktops	Largura máxima do container (1200px)

💡 Boas práticas de breakpoints

1. Prefira min-width (mobile-first) a max-width (desktop-first).
2. Use variáveis CSS para centralizar os breakpoints em um único lugar.
3. Teste no navegador redimensionando a janela — não só em device modes.
4. O conteúdo dita o breakpoint, não o dispositivo.

4.4 Viewport Meta Tag — Pré-requisito Obrigatório

Sem esta meta tag, browsers mobile renderizam a página com 980px de largura e aplicam zoom out, ignorando suas media queries de largura menor. Ela é o pré-requisito de toda responsividade:

```
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- width=device-width: largura do viewport = largura do dispositivo -->
  <!-- initial-scale=1.0: sem zoom inicial -->
</head>
```

5. Unidades de Medida CSS

5.1 Absolutas vs Relativas

Unidades absolutas têm valor fixo, independente do contexto. Unidades relativas calculam seu valor a partir de algo — tamanho do elemento pai, tamanho da fonte raiz, dimensão do viewport. Em design responsivo, unidades relativas são preferidas porque se adaptam automaticamente ao contexto.

Unidade	Tipo	Referência	Uso recomendado
px	Absoluta	1 pixel CSS (≈1 pixel físico em tela 1x)	Bordas, shadows, valores mínimos/máximos precisos
cm / mm	Absoluta	Centímetro / milímetro físico	Impressão; evitar para telas
pt	Absoluta	1/72 de polegada	Tipografia impressa; evitar para telas
em	Relativa	Tamanho da fonte do elemento PAI	Padding/margin relativos ao texto do componente
rem	Relativa	Tamanho da fonte do elemento RAIZ (html)	Tipografia escalável globalmente — preferência
vw	Relativa	1% da largura do viewport	Tipografia fluida, seções full-width
vh	Relativa	1% da altura do viewport	Seções hero, modais full-height
%	Relativa	Percentual do elemento pai	Larguras fluidas de layout
ch	Relativa	Largura do caractere '0' da fonte	Largura ideal de colunas de texto (45–75ch)
clamp()	Função	min, valor preferencial, max	Tipografia e espaçamentos verdadeiramente fluidos

5.2 rem vs em — O Debate mais Frequente

A diferença entre rem e em é crucial e fonte de muita confusão. rem sempre se refere ao tamanho de fonte do elemento html — é previsível. em se refere ao elemento pai imediato — pode acumular em composições aninhadas:

```
html { font-size: 16px; } /* base padrão dos browsers */

/* rem – sempre relativo a 16px */
h1 { font-size: 2rem; } /* 32px sempre */
.btn { padding: 0.75rem 1.5rem; } /* 12px 24px – consistente */

/* em – relativo ao pai – pode compor/escalar */
.card { font-size: 0.875rem; } /* 14px */
.card .title { font-size: 1.25em; } /* 1.25 × 14 = 17.5px */
.card .meta { font-size: 0.857em; } /* 0.857 × 14 = 12px */

/* clamp() – fluido entre um mínimo e um máximo */
h1 { font-size: clamp(1.5rem, 4vw, 3rem); }
/* mínimo: 24px | cresce com o viewport | máximo: 48px */
```

⚡ Dica de arquitetura CSS

Padrão amplamente adotado no mercado:

- Use rem para font-size, padding e margin globais.
- Use em para padding/margin dentro de componentes (escala com o próprio componente).
- Use px para border, border-radius, box-shadow e valores de 1-2px.
- Use % ou fr para larguras de layout.
- Use vw/vh para seções full-screen e tipografia fluida.

5.3 Boas Práticas de Tipografia Responsiva

```
/* Reset de fonte na raiz – facilita cálculo mental */
/* 1rem = 10px, 1.6rem = 16px */
html { font-size: 62.5%; }
body { font-size: 1.6rem; } /* 16px */

/* Tipografia fluida com clamp – sem media queries */
h1 { font-size: clamp(2.4rem, 5vw, 4.8rem); }
h2 { font-size: clamp(2rem, 3.5vw, 3.2rem); }
p { font-size: clamp(1.4rem, 2vw, 1.8rem); }

/* Largura máxima para conforto de leitura */
article { max-width: 70ch; margin-inline: auto; }
```

6. Síntese e Ponte para a Prática

Você acabou de aprender as quatro peças fundamentais do layout moderno e responsivo: Flexbox para distribuição unidimensional, CSS Grid para estruturas bidimensionais, Media

Queries para adaptar ao contexto do dispositivo, e unidades relativas para criar proporções que escalam organicamente.

A teoria ganha vida real quando você aplica essas ferramentas juntas em um projeto. Na etapa prática a seguir, você vai pegar o layout do projeto da dupla — estático até agora — e transformá-lo em uma interface genuinamente responsiva, que funciona com elegância em qualquer dispositivo. Esse é o tipo de trabalho que os recrutadores procuram no portfólio de um desenvolvedor front-end.

7. Glossário

Termo	Definição
Flex Container	Elemento HTML que recebe display: flex, tornando seus filhos diretos flex items com comportamento flexível.
Flex Item	Filho direto de um flex container; responde às propriedades de alinhamento e distribuição do Flexbox.
Main Axis	Eixo principal do Flexbox, definido por flex-direction. É o eixo ao longo do qual os itens são distribuídos.
Cross Axis	Eixo perpendicular ao main axis no Flexbox. O alinhamento neste eixo é controlado por align-items e align-self.
justify-content	Propriedade do flex/grid container que distribui itens ao longo do eixo principal.
align-items	Propriedade do flex container que alinha itens no eixo cruzado.
CSS Grid	Módulo CSS que implementa layout bidimensional (linhas e colunas simultâneas) em um único container.
fr (fraction)	Unidade exclusiva do CSS Grid que representa uma fração do espaço disponível após alocar tamanhos fixos.
grid-template-areas	Propriedade CSS que nomeia regiões da grade usando texto, criando um mapa visual do layout.
Media Query	Condicional CSS (@media) que aplica estilos apenas quando características do dispositivo (largura, orientação, etc.) são satisfeitas.
Breakpoint	Ponto de largura de viewport onde o layout muda de configuração através de uma media query.
Mobile-First	Estratégia de CSS onde os estilos base são escritos para dispositivos móveis e media queries com min-width adicionam complexidade para telas maiores.
RWD	Responsive Web Design — abordagem de design que usa media queries, grids fluidos e imagens flexíveis para criar layouts adaptáveis.
rem	Unidade CSS relativa ao font-size do elemento raiz (html). Previsível e global — recomendada para tipografia.
em	Unidade CSS relativa ao font-size do elemento pai. Útil para escalar componentes proporcionalmente ao seu contexto.
vw / vh	Viewport Width e Viewport Height — unidades relativas a 1% das dimensões do viewport.
clamp()	Função CSS com três parâmetros: clamp(min, preferred, max). Retorna o valor preferencial limitado pelo mínimo e máximo.
auto-fit / auto-fill	Palavras-chave do CSS Grid usadas com repeat() para criar grades responsivas sem media queries.
viewport	Área visível do browser. A meta tag viewport controla como o browser renderiza a página em dispositivos móveis.
gap	Propriedade CSS shorthand para row-gap e column-gap — espaçamento entre itens de flex containers e grid containers.

8. Rubrica de Avaliação

Critério	Peso	Insuficiente (0–4)	Regular (5–6)	Bom (7–8)	Excelente (9–10)
Funcionalidade Responsiva	35%	Layout quebra em um ou mais breakpoints; overflow horizontal presente; elementos sobrepostos.	Layout funcional em pelo menos 2 dos 3 breakpoints; pequenos problemas visuais.	Layout funcional nos 3 breakpoints; sem overflow; imagens e inputs responsivos.	Layout fluido e elegante; transições suaves; funciona em orientação landscape e portrait; desafio extra implementado.
Qualidade do CSS	25%	CSS caótico, sem organização; uso de !important excessivo; larguras fixas em px; sem variáveis.	CSS funcional mas com repetições; alguns valores hardcoded; sem tokens de design.	CSS organizado, variáveis CSS usadas, sem repetições desnecessárias; mobile-first aplicado.	Arquitetura CSS exemplar: variáveis, mobile-first rigoroso, sem código morto, comentários úteis, propriedades shorthand corretas.
Aplicação dos Conceitos	25%	Não usa Flexbox nem Grid; layout baseado em float ou table; sem media queries.	Usa Flexbox ou Grid mas de forma superficial; media queries com poucos breakpoints ou desktop-first.	Aplica Flexbox para componentes e Grid para layout macro; 3 breakpoints corretos; unidades relativas usadas.	Domínio pleno de Flexbox, Grid e Media Queries; usa clamp(), fr, rem; auto-fit/minmax na grade de cards.
Entrega no Prazo	15%	Entregue com atraso superior a 1 semana ou não entregue.	Entregue com atraso de até 1 semana.	Entregue no prazo com repositório organizado.	Entregue no prazo; repositório com commits descritivos; README com screenshots e instruções de execução.

9. Referências Bibliográficas

MEYER, Eric A.; WEYL, Estelle. CSS: The Definitive Guide — Layout. 5. ed. Sebastopol: O'Reilly Media, 2023.

CEDERHOLM, Dan. CSS3 for Web Designers. 2. ed. New York: A Book Apart, 2015.

MDN WEB DOCS. Flexbox. Mozilla Foundation, 2024. Disponível em: https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Flexbox. Acesso em: abr. 2025.

MDN WEB DOCS. CSS Grid Layout. Mozilla Foundation, 2024. Disponível em: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout. Acesso em: abr. 2025.

W3C. CSS Flexible Box Layout Module Level 1. World Wide Web Consortium, 2018. Disponível em: <https://www.w3.org/TR/css-flexbox-1/>. Acesso em: abr. 2025.

W3C. CSS Grid Layout Module Level 1. World Wide Web Consortium, 2020. Disponível em: <https://www.w3.org/TR/css-grid-1/>. Acesso em: abr. 2025.

MARCOTTE, Ethan. Responsive Web Design. A List Apart, v. 306, 2010. Disponível em: <https://alistapart.com/article/responsive-web-design/>. Acesso em: abr. 2025.

SMITH, Rachel Andrew. The New CSS Layout. New York: A Book Apart, 2017.

CSS-TRICKS. A Complete Guide to Flexbox. 2023. Disponível em: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>. Acesso em: abr. 2025.

CSS-TRICKS. A Complete Guide to CSS Grid. 2023. Disponível em: <https://css-tricks.com/snippets/css/complete-guide-grid/>. Acesso em: abr. 2025.

GOOGLE. Responsive Web Design Basics. Web Fundamentals, 2023. Disponível em: <https://web.dev/responsive-web-design-basics/>. Acesso em: abr. 2025.

WEYL, Estelle. Viewport Units: vw, vh, vmin, vmax. CSS WG Blog, 2012. Disponível em: <https://www.w3.org/blog/CSS/2012/09/18/>. Acesso em: abr. 2025.